

Q1) Consider the problem of rod cutting discussed in class. Dry run dynamic programming algorithm on following input for rod length 7. Show all computations and write values in the given array C[n] for memoization. [5 Marks]

Length (i)	1	2	3	4	5	6	7
Price P[i]	2	5	7	12	13	16	17

$$C[0] = 0$$

$$C[n] = \max_{1 \leq i \leq n} (P[i] + C[n-i])$$

Solution:

n	1	2	3	4	5	6	7
C[n]	2	5	7	12	14	17	19

$$C[1] = P[1] = 2$$

$$C[2] = \max \begin{cases} P[1] + C[1] = 2 + 2 = 4 \\ P[2] = 5 \end{cases}$$

$$C[2] = 5$$

$$C[3] = \max \begin{cases} P[1] + C[2] = 2 + 5 = 7 \\ P[2] + C[1] = 5 + 2 = 7 \\ P[3] = 7 \end{cases}$$

$$C[3] = 7$$

$$C[4] = \max \begin{cases} P[1] + C[3] = 2 + 7 = 9 \\ P[2] + C[2] = 5 + 5 = 10 \\ P[3] + C[1] = 7 + 2 = 9 \\ P[4] = 12 \end{cases}$$

$$C[4] = 12$$

$$C[5] = \max \begin{cases} P[1] + C[4] = 2 + 12 = 14 \\ P[2] + C[3] = 5 + 7 = 12 \\ P[3] + C[2] = 7 + 5 = 12 \\ P[4] + C[1] = 12 + 2 = 14 \\ P[5] = 13 \end{cases}$$

$$C[5] = 14$$

$$C[6] = \max \begin{cases} P[1] + C[5] = 2 + 14 = 16 \\ P[2] + C[4] = 5 + 12 = 17 \\ P[3] + C[3] = 7 + 7 = 14 \\ P[4] + C[2] = 12 + 5 = 17 \\ P[5] + C[1] = 13 + 2 = 15 \\ P[6] = 16 \end{cases}$$

$$C[6] = 17$$

$$C[7] = \max \begin{cases} P[1] + C[6] = 2 + 17 = 19 \\ P[2] + C[5] = 5 + 14 = 19 \\ P[3] + C[4] = 7 + 12 = 19 \\ P[4] + C[3] = 12 + 7 = 19 \\ P[5] + C[2] = 13 + 5 = 18 \\ P[6] + C[1] = 16 + 2 = 18 \\ P[7] = 17 \end{cases}$$

$$C[7] = 19$$

Q2) Consider the following recursive algorithm. [1+4 = 5 Marks]

/ m and n are lengths of char arrays X and Y respectively */*

```
int Function( char *X, char *Y, int m, int n )  
{  
    if (m == 0 || n == 0)  
        return 0;  
    if (X[m-1] == Y[n-1])  
        return 1 + Function (X, Y, m-1, n-1);  
    else  
        return max (Function (X, Y, m, n-1) +1, Function (X, Y, m-1, n) +1);  
}
```

(a) What is time complexity of above algorithm? Show all working

Exponential Time :

$$O(2^{m+n})$$

(b) Convert the recursive code given above into bottom up iterative dynamic programming algorithm.
Write its time complexity.

```
int Function (char * X, char * Y, int m, int n)
```

```
{
```

```
    for (i=1 to n)
```

```
        A[i][0] = 0
```

```
    for (i=1 to m)
```

```
        A[0][i] = 0
```

```
    for (i=1 to n)
```

```
        for (i=1 to m) -
```

```
            if (X[m-1] == Y[n-1])
```

```
                A[i][j] = A[i-1][j-1] + 1
```

```
            else
```

```
                A[i][j] = max (A[i][j-1], A[i-1][j])
```

FAST School of Computing

Page 2

```
    Return A[m][n]
```

```
}
```

Time complexity = ~~124~~ $\Theta(m \times n)$ 0245

Q3) You are given n integers in the range $[1, m]$ and a target T . Find a subset of the n integers whose sum is as close to T as possible without exceeding it. For example,

$T = 10$,

Set of n integers = $\{3, 5, 4\}$

All possible subsets = $\{3\}, \{5\}, \{4\}, \{3, 5\}, \{3, 4\}, \{5, 4\}, \{3, 5, 4\}$

Optimal Solution = subset = $\{5, 4\}$ // The sum is 9, any other subset sum will either exceed 10 or will be less than 9.

Consider following greedy approach for solving this problem. Sort number in descending order. Start adding numbers in subset starting from largest number. Terminate at the number for which the sum exceeds T . Prove this greedy strategy is not optimal by giving counter example. [5 Marks]

Solution:

$T = 6$

Set = $\{1, 4, 3\}$ $\rightarrow$ Sort in decreasing order $\{4, 3, 1\}$

Subset given by the Greedy algorithm = $\{4\}$

Sum $\rightarrow 4$

Subset given by the optimal solution = $\{4, 1\}$

Sum $\rightarrow 5$

Q4) This question is about maximum subarray sum problem. We are given an integer array $A[0..n]$ and we have to find the subarray $A[i..j]$ such that the sum of the integers $A[i] + A[i+1] + \dots + A[j-1]$ is maximal. If $M[j]$ holds the maximum sum for any segment $A[i..j]$ ending in $j-1$ how can we efficiently compute the maximum sum for any segment $A[i.....j+1]$ ending in j ? In other words, write the expression (recursive equation) for defining the value of $M[j+1]$.
[Hint: you do not need a loop or pseudocode.] [3 Marks]

$$M[j+1] = \max \{ M[j] + A[j], A[j] \}$$